



FPT UNIVERSITY

**CHỦ ĐỀ: MUSIC STREAMING
PLAYLIST MANAGER**

Môn học: CSD201

Giảng viên: Đỗ Đức Hào

Nhóm 3

Nguyễn Duy Hoàng
Nguyễn Thành Sơn
Nhiều Sĩ Luân
Bùi Văn Nghĩa

SE201697
SE201777
SE201956
SE201912

Hồ Chí Minh, 24/05/2026
Học kỳ:SP_2026

1. Phân rã Cấu trúc dữ liệu

- **Thực thể Song (Data Node - Thể hiện tính Đóng gói):**
 - Thay vì một struct đơn thuần, Song là một đối tượng chứa toàn bộ dữ liệu (metadata) của bài hát.
 - Các thuộc tính được giấu kín (private) để bảo vệ dữ liệu, và chỉ được truy cập qua các phương thức *getter/setter*. Đối tượng Song sẽ tự giữ một tham chiếu đến đối tượng Song tiếp theo (next).
- **Thực thể CircularLinkedList (Phục vụ tính năng Repeat):**
 - Đây là một Class độc lập, tự quản lý các logic thêm/xóa/duyệt các đối tượng Song.
 - Để thể hiện **tính Đa hình (Polymorphism)**, lớp này có thể *implements* một giao diện (Interface) ví dụ như *IPlaylistStructure*. Điều này giúp hệ thống linh hoạt: nếu sau này nhóm bạn muốn đổi sang danh sách liên kết đôi (Doubly Linked List), phần code điều khiển nhạc ở ngoài sẽ không cần thay đổi.
 - Lợi thế: Tham chiếu của Node cuối cùng luôn trở về Node head, giúp thao tác "Next" khi hết danh sách diễn ra hoàn toàn tự động và liền mạch.
- **Thực thể HistoryStack (Phục vụ tính năng Previous):**
 - Ngăn xếp chịu trách nhiệm quản lý lịch sử theo nguyên lý LIFO.
 - Bạn có thể xây dựng một Class HistoryStack riêng biệt để đảm bảo **tính Trừu tượng (Abstraction)**. Mặc dù bên dưới bạn có thể dùng một mảng (Song[]) hoặc tận dụng `java.util.ArrayList` để lưu trữ, nhưng phần giao diện phơi bày ra ngoài chỉ là các hàm `push()` (khi bài hát bắt đầu) và `pop()` (khi bấm Previous) với độ phức tạp $O(1)$.
- **Mảng động (Phục vụ tính năng Shuffle):**
 - Linked List thao tác chậm với việc lấy phần tử ngẫu nhiên do phải duyệt tuần tự $O(n)$.
 - Với Java, bạn có thể tận dụng một cấu trúc mảng động (như `ArrayList<Song>`) để sao chép tham chiếu của các bài hát hiện có. Khi bật Shuffle, hệ thống chỉ cần gọi hàm `random` để lấy index bất kỳ trong khoảng bộ nhớ liên tục với tốc độ $O(1)$.

2. Phân rã Module thành các Lớp Chức năng (Functional Objects)

Hệ thống sẽ được chia thành các Object tương tác với nhau, tuân thủ nguyên lý **Đơn trách nhiệm (Single Responsibility Principle)** – mỗi Class chỉ làm đúng một nhiệm vụ cốt lõi.

- **Module Quản lý Dữ liệu (PlaylistManager):**
 - **Trách nhiệm:** Tập trung xử lý tính toán vẹn của cấu trúc dữ liệu.

- **Hành vi:** Đóng vai trò như một wrapper bọc ngoài CircularLinkedList. Chịu trách nhiệm thêm bài hát mới, xóa bài hát bị lỗi, hoặc tái tạo lại danh sách. Lớp này không quan tâm đến việc bài hát đang phát hay dừng.
- **Module Điều khiển Phát nhạc (PlaybackController):**
 - **Trách nhiệm:** Đây là "bộ não" của trình phát nhạc. Lớp này thể hiện mối quan hệ **Thành phần (Composition)** bằng cách "sở hữu" đối tượng PlaylistManager và HistoryStack.
 - **Hành vi:** Xử lý các logic điều hướng phức tạp.
 - Khi gọi next(): Nó lấy bài hát tiếp theo từ cấu trúc vòng, đồng thời gọi HistoryStack.push() để lưu lại bài cũ.
 - Khi gọi previous(): Nó gọi HistoryStack.pop() để khôi phục trạng thái.
 - Kiểm soát các biến trạng thái (boolean isShuffle, boolean isRepeat) để chuyển hướng luồng dữ liệu sang Array hoặc Linked List.
- **Module Giao diện (UserInterface):**
 - **Trách nhiệm:** Giao tiếp với người dùng cuối thông qua Console (sử dụng Scanner) hoặc Java Swing/JavaFX nếu làm giao diện đồ họa.
 - **Hành vi:** Nhận lệnh (phím bấm, click chuột) và truyền tín hiệu tương ứng cho PlaybackController xử lý, sau đó in ra kết quả ("Đang phát: ...", "Chế độ Shuffle: Bật").
-

3. Áp dụng Nhận dạng Mẫu (Pattern Recognition) trong Tổ chức Dữ liệu

Quá trình Nhận dạng Mẫu (Pattern Recognition) giúp xác định các mô hình tổ chức dữ liệu và kiến trúc phần mềm phù hợp nhất để giải quyết từng bài toán cụ thể trong hệ thống trình phát nhạc. Dựa trên các thực thể và module đã được phân rã ở bước trước (Decomposition), hệ thống tiến hành nhận diện các quy luật xử lý dữ liệu đặc thù nhằm lựa chọn cấu trúc bộ nhớ và mẫu thiết kế tối ưu nhất cho từng chức năng.

3.1. Các Mẫu Tổ chức Dữ liệu Cốt lõi (Core Data Organization Patterns)

Mô hình Vòng lặp Tuần tự Liên tục (Cyclic Sequential Pattern)

- **Vấn đề:** Trình phát nhạc cần hỗ trợ chế độ phát lặp lại toàn bộ danh sách (Repeat All) một cách liên tục và vô tận. Nếu sử dụng cấu trúc mảng tuyến tính thông thường, hệ thống phải liên tục đặt các câu lệnh kiểm tra điều kiện rẽ nhánh (ví dụ: kiểm tra xem chỉ số hiện tại đã vượt qua kích thước mảng hay chưa) mỗi khi chuyển bài, làm tăng rủi ro lỗi vượt giới hạn chỉ số và tăng độ phức tạp xử lý không cần thiết.
- **Mẫu áp dụng:** Hệ thống áp dụng cấu trúc **Circular Linked List** (Danh sách liên kết vòng). Mỗi thực thể Song sẽ tự giữ một tham chiếu đến bài hát tiếp theo thông qua thuộc tính next. Đặc biệt, Node cuối cùng sẽ liên kết ngược trở lại Node đầu tiên, tạo thành một vòng lặp khép kín tự nhiên về mặt cấu trúc vật lý.
- **Lợi ích đạt được:**
 - Thao tác chuyển bài next() luôn được thực hiện với độ phức tạp thời gian là $O(1)$.
 - Loại bỏ hoàn toàn các câu lệnh kiểm tra điều kiện ở biên danh sách, giúp mã nguồn sạch và dễ bảo trì hơn.
 - Luồng phát nhạc diễn ra liên tục, tự động và liền mạch.

Mô hình Phục hồi Trạng thái (LIFO / Undo Pattern)

- **Vấn đề:** Chức năng lùi bài (Previous) không đơn thuần là lùi lại một vị trí trong danh sách, mà phải phản ánh chính xác bài hát thực tế mà người dùng vừa nghe trước đó. Điều này đặc biệt quan trọng khi chế độ trộn bài (Shuffle) đang được bật, việc lùi vị trí tuyến tính trên danh sách phát gốc sẽ phá vỡ hoàn toàn trải nghiệm lịch sử của người dùng.
- **Mẫu áp dụng:** Hệ thống sử dụng cấu trúc **Stack** (Ngăn xếp) để quản lý lịch sử phát nhạc, hoạt động độc lập với danh sách phát chính. Mỗi khi một bài hát mới bắt đầu được phát, tham chiếu của bài hát đó sẽ được push() vào HistoryStack. Khi người dùng nhấn yêu cầu lùi bài, hệ thống gọi lệnh pop() để khôi phục lại trạng thái gần nhất.
- **Lợi ích đạt được:**
 - Hành vi dữ liệu tuân thủ nghiêm ngặt nguyên lý LIFO (Last-In-First-Out), phù hợp với mẫu thiết kế Undo chuẩn mực trong kỹ nghệ phần mềm.
 - Đảm bảo truy xuất lịch sử phát nhạc chính xác tuyệt đối ngay cả khi luồng phát đang bị xáo trộn bởi Shuffle.

- Các thao tác thêm vào (push) và lấy ra (pop) đều đạt hiệu năng tối đa với độ phức tạp $O(1)$.

Mô hình Truy cập Ngẫu nhiên (Random Access Pattern)

- **Vấn đề cốt lõi:** Tính năng trộn bài (Shuffle) yêu cầu hệ thống phải chọn ngẫu nhiên một bài hát bất kỳ trong toàn bộ playlist. Nếu tiếp tục sử dụng cấu trúc Linked List, hệ thống sẽ buộc phải duyệt tuần tự từ đầu danh sách để tìm đến vị trí ngẫu nhiên đó, dẫn đến độ phức tạp thời gian là $O(n)$. Với các danh sách nhạc có quy mô lớn, điều này sẽ tạo ra nút thắt cổ chai nghiêm trọng về hiệu năng.
- **Mẫu áp dụng:** Sử dụng Mảng động (ArrayList<Song>) để lưu bản sao tham chiếu. Thay vì chọn ngẫu nhiên từng lượt, hệ thống dùng thuật toán **Fisher-Yates** để xáo trộn toàn bộ mảng một lần duy nhất (in-place). Sau đó, hệ thống chỉ cần duyệt tuần tự trên mảng đã xáo trộn này.
- **Lợi ích đạt được:**
 - Tối ưu hóa tốc độ chuyển bài trong chế độ Shuffle xuống mức $O(1)$.
 - Thuật toán Fisher-Yates tiết kiệm tài nguyên với thời gian $O(n)$ và không gian $O(1)$.
 - Đảm bảo không có bài hát nào bị phát lặp lại trong cùng một chu kỳ trộn bài.

Mô hình Tìm kiếm Phân cấp (Hierarchical Search Pattern)

- **Vấn đề cốt lõi:** Khi thư viện âm nhạc mở rộng, người dùng phát sinh nhu cầu truy vấn một bài hát cụ thể dựa trên tiêu đề. Nếu hệ thống tái sử dụng cấu trúc Mảng tĩnh hoặc Danh sách liên kết để thực hiện tác vụ này, con trỏ buộc phải duyệt tuần tự qua từng phần tử (Linear Search) với độ phức tạp thời gian là $O(n)$. Đối với một kho dữ liệu lớn, thao tác này tạo ra độ trễ đáng kể, làm giảm trải nghiệm người dùng và lãng phí tài nguyên xử lý.
- **Mẫu áp dụng:** Hệ thống tích hợp thêm cấu trúc **Cây Nhị Phân Tìm Kiếm (Binary Search Tree - BST)**, hoạt động như một bộ chỉ mục song song với danh sách phát chính. Mỗi thực thể bài hát khi được thêm vào hệ thống sẽ được phân bổ vào các nút (node) của cây dựa trên nguyên tắc so sánh chuỗi phân cấp: Nút chứa tiêu đề đứng trước theo bảng chữ cái sẽ rẽ sang nhánh trái, và nút chứa tiêu đề đứng sau sẽ rẽ sang nhánh phải.
- **Lợi ích đạt được:**
 - Tối ưu hóa triệt để thao tác tìm kiếm bài hát. Thuật toán loại bỏ được một nửa số lượng phần tử cần kiểm tra ở mỗi bước rẽ nhánh, giúp rút ngắn độ phức tạp thời gian trung bình xuống mức lý tưởng là $O(\log n)$.
 - Cấu trúc cây tự động duy trì thứ tự của dữ liệu. Bằng cách áp dụng thuật toán duyệt cây In-order, hệ thống có thể trích xuất ngay lập tức danh sách toàn bộ bài hát theo thứ tự từ khóa A-Z mà không cần phải tiêu tốn thêm tài nguyên cho các thuật toán sắp xếp bên ngoài.

3.2. Các Mẫu Luồng Dữ liệu (Behavioral Data Flow Patterns)

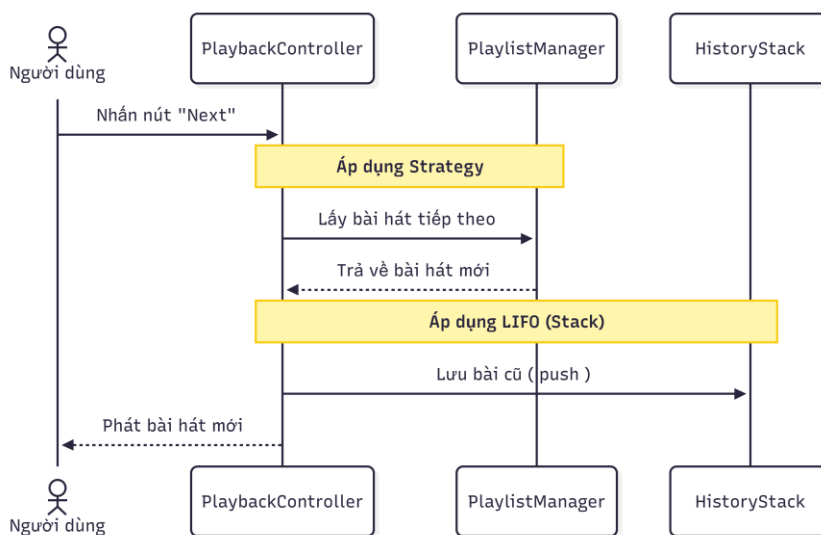
Thông qua quá trình phân tích hành vi xử lý dữ liệu của người dùng, hệ thống không sử dụng một cấu trúc duy nhất mà linh hoạt ánh xạ từng chức năng với một pattern luồng dữ liệu riêng biệt để tối ưu hóa khả năng vận hành:

- **Chức năng Repeat Playlist (Mẫu Cyclic Traversal):** Áp dụng luồng dữ liệu tịnh tiến vòng tròn. Hệ thống không thiết lập điểm kết thúc vật lý của mảng, giúp duy trì trạng thái phát nhạc vĩnh viễn mà không cần tái khởi tạo bộ đếm hay cấp phát lại bộ nhớ.
- **Chức năng Previous Song (Mẫu LIFO Recovery):** Áp dụng luồng dữ liệu phục hồi trạng thái. Hệ thống ưu tiên tính chính xác của lịch sử thao tác thực tế thay vì phụ thuộc vào vị trí vật lý của bài hát trong danh sách gốc.
- **Chức năng Shuffle Song (Mẫu Random Access):** Áp dụng luồng dữ liệu phân tán. Mô hình này cho phép con trỏ hệ thống nhảy cóc đến bất kỳ địa chỉ bộ nhớ nào trong mảng tĩnh với tốc độ phản hồi tức thời, loại bỏ sự rườm rà của việc duyệt qua các Node trung gian.
- **Chức năng Next Song (Mẫu Sequential Traversal):** Áp dụng luồng dữ liệu tuần tự chuẩn mực, chuyển tiếp con trỏ một cách an toàn và có kiểm soát sang vùng nhớ kế tiếp dọc theo danh sách liên kết.

3.3. Trực quan hóa Tương tác Động qua Sơ đồ Tuần tự (Sequence Diagrams)

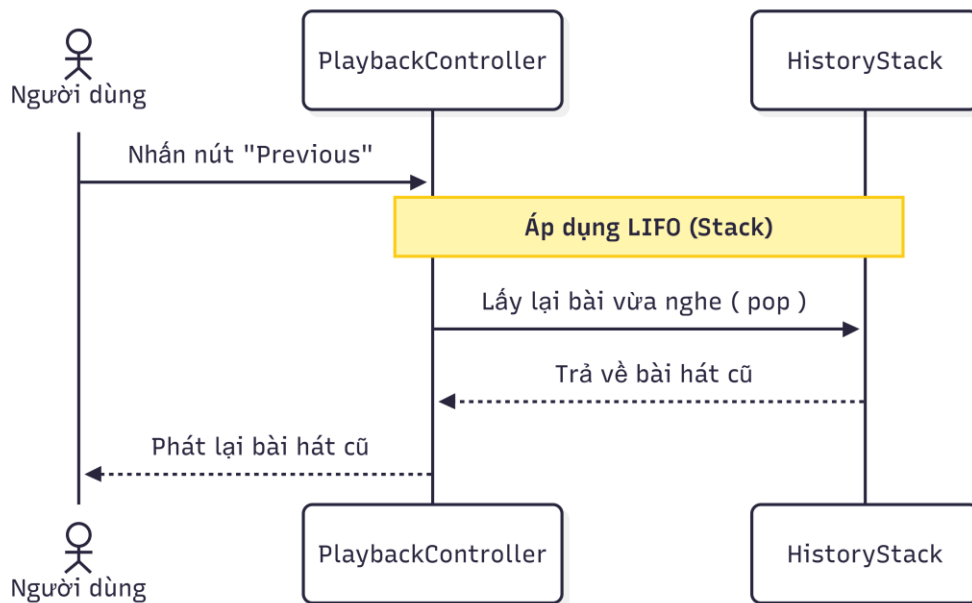
Sơ đồ 1: Luồng xử lý chuyển bài hát kế tiếp (Next Track) Luồng xử lý này thể hiện sự kết hợp giữa mẫu Facade (giao tiếp qua PlaybackController), mẫu Strategy (gọi qua PlaylistManager), và mẫu LIFO (lưu vết vào HistoryStack).

- Khi người dùng kích hoạt lệnh, PlaybackController đóng vai trò điều phối trung tâm. Nó không trực tiếp can thiệp vào cách lấy bài mà ủy nhiệm cho PlaylistManager để lấy thực thể bài hát tiếp theo. Sau khi nhận lại dữ liệu, hệ thống ngay lập tức đẩy trạng thái cũ vào ngăn xếp lịch sử bằng hàm push() trước khi cập nhật bài hát mới lên giao diện người dùng.



Sơ đồ 2: Luồng xử lý quay lại bài hát trước (Previous Track) Luồng xử lý này chứng minh tính chuẩn xác của mẫu Phục hồi Trạng thái khi khôi phục dữ liệu trực tiếp từ ngăn xếp lịch sử mà không phụ thuộc vào thứ tự tuyến tính của danh sách phát.

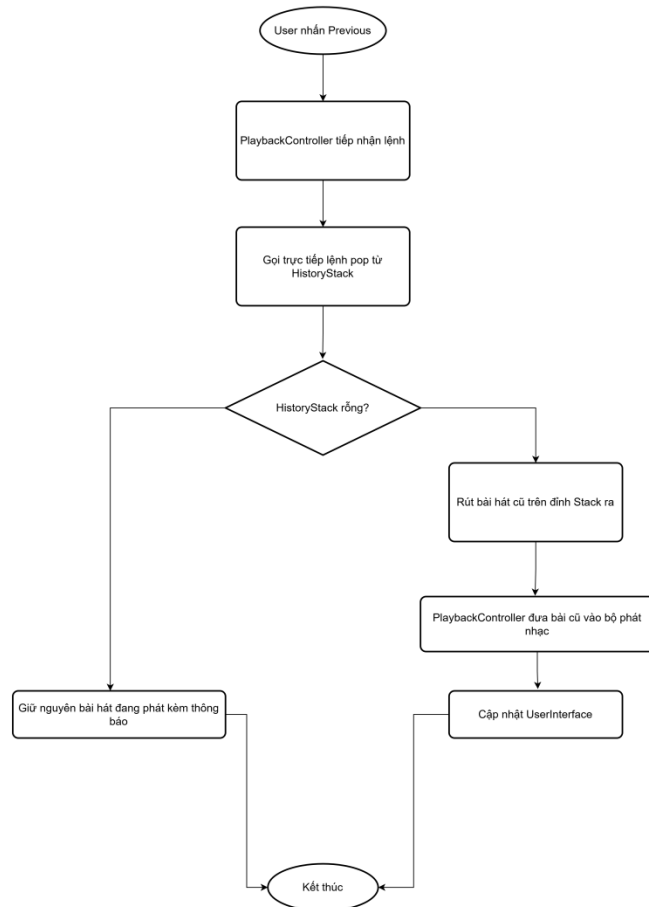
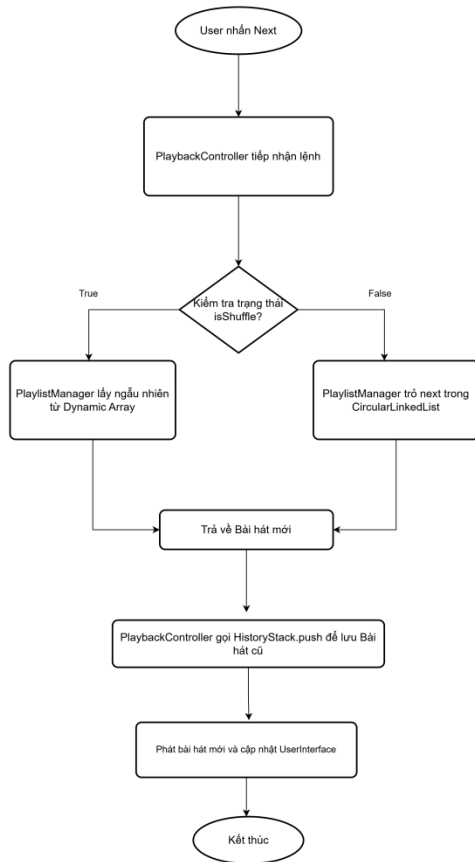
- Đối với thao tác quay lại bài trước, luồng dữ liệu được tối giản hóa tối đa nhằm đạt hiệu năng cao nhất. PlaybackController bỏ qua hoàn toàn module quản lý danh sách phát và thực hiện lệnh pop() trực tiếp lên HistoryStack. Phần tử nằm ở trên cùng của ngăn xếp (bài hát vừa phát gần nhất) được rút ra và đưa thẳng vào bộ xử lý phát nhạc, đảm bảo tính chính xác tuyệt đối về mặt lịch sử trải nghiệm.



4. Flowchart

Flowchart mô tả luồng xử lý của hai thao tác chính trong hệ thống Music Streaming Playlist Manager: chuyển bài kế tiếp (Next Song) và quay lại bài trước (Previous Song). Mỗi luồng thể hiện cách các module PlaybackController, PlaylistManager và HistoryStack phối hợp với nhau để xử lý lệnh từ người dùng.

4.1. Flowchart: Next Song & Previous Song



Flowchart Next Song:

- Bắt đầu khi người dùng nhấn nút Next.
- PlaybackController tiếp nhận lệnh và kiểm tra trạng thái isShuffle.
- Nếu isShuffle = True: PlaylistManager lấy ngẫu nhiên một bài hát từ Dynamic Array với tốc độ $O(1)$.
- Nếu isShuffle = False: PlaylistManager trở đến node kế tiếp trong CircularLinkedList.
- Bài hát mới được trả về, PlaybackController gọi HistoryStack.push() để lưu bài hát cũ vào lịch sử.
- Phát bài hát mới và cập nhật UserInterface. Kết thúc.

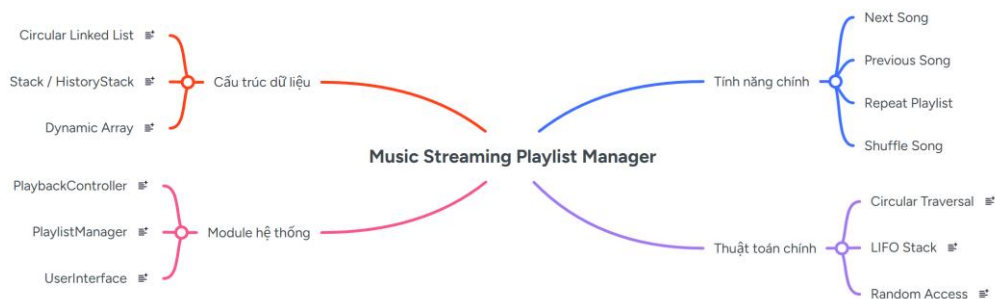
Flowchart Previous Song:

- Bắt đầu khi người dùng nhấn nút Previous.
- PlaybackController tiếp nhận lệnh và gọi trực tiếp lệnh pop() từ HistoryStack.
- Hệ thống kiểm tra HistoryStack có rỗng không.
- Nếu Stack rỗng: giữ nguyên bài hát đang phát và hiển thị thông báo cho người dùng.
- Nếu Stack không rỗng: rút bài hát cũ trên đỉnh Stack ra, PlaybackController đưa vào bộ phát nhạc.
- Cập nhật UserInterface và kết thúc.

5. Mind Map

Mind Map tổng quan hóa toàn bộ kiến trúc và các thành phần cốt lõi của hệ thống Music Streaming Playlist Manager. Sơ đồ được tổ chức theo 4 nhánh chính từ chủ đề trung tâm:

- Tính năng chính: Next Song, Previous Song, Repeat Playlist, Shuffle Song.
- Cấu trúc dữ liệu: Circular Linked List (Repeat), Stack/HistoryStack (Previous), Dynamic Array (Shuffle).
- Thuật toán chính: Circular Traversal $O(1)$, LIFO Stack $push()/pop()$, Random Access $O(1)$.
- Module hệ thống: PlaybackController (bộ não), PlaylistManager (quản lý danh sách), UserInterface (giao diện).



Mỗi nhánh trong Mind Map liên kết chặt chẽ với nhau: tính năng yêu cầu cấu trúc dữ liệu phù hợp, cấu trúc dữ liệu quyết định thuật toán áp dụng, và các thuật toán được triển khai trong các module tương ứng của hệ thống.

6. Research Question (RQ):

6.1. Câu hỏi nghiên cứu 1 (RQ1): Tối ưu hóa cấu trúc dữ liệu cho luồng điều hướng tuần hoàn và lưu vết lịch sử phát nhạc.

- **Câu hỏi nghiên cứu:** Việc kết hợp đồng thời cấu trúc Danh sách liên kết vòng (Circular Linked List) và Ngăn xếp (Stack) giúp tối ưu hóa hiệu suất bộ nhớ và đảm bảo tính chính xác của luồng dữ liệu như thế nào khi thực hiện các thao tác chuyển bài tuần hoàn (Next) và lùi bài theo lịch sử thực tế (Previous)?
- **Giải thích nội dung:** Trong một trình phát nhạc tiêu chuẩn, tính năng phát lại toàn bộ playlist đòi hỏi con trỏ hệ thống phải tịnh tiến liên tục dọc theo bộ nhớ. Nếu dùng mảng tuyến tính thông thường, hệ thống sẽ vấp phải các câu lệnh kiểm tra điều kiện biên rườm rà ở phần tử cuối cùng. Việc nhận dạng mẫu sang **Circular Linked List** giúp cho thao tác chuyển bài kế tiếp next() diễn ra tuần hoàn tự nhiên và đạt độ phức tạp thời gian lý tưởng $O(1)$.

6.2. Câu hỏi nghiên cứu 2 (RQ2): Giải quyết xung đột hiệu năng giữa truy xuất ngẫu nhiên (Shuffle) và duyệt tuần tự (Sequential Traversal).

- **Câu hỏi nghiên cứu:** Làm thế nào để tích hợp tính năng phát ngẫu nhiên (Shuffle) vào hệ thống đang vận hành bằng Danh sách liên kết (Linked List) mà không làm suy giảm hiệu năng hệ thống và không làm xáo trộn trật tự sắp xếp ban đầu của playlist?
- **Giải thích nội dung:** Bản chất cấu trúc dữ liệu Danh sách liên kết (Linked List) thao tác rất chậm với các yêu cầu truy xuất phần tử ngẫu nhiên do con trỏ phải duyệt tuần tự từ nút đầu tiên (Head) qua các nút trung gian, đẩy độ phức tạp thời gian lên tới $O(n)$. Đối với các playlist có quy mô dữ liệu lớn, việc "nhảy cóc" liên tục khi bật Shuffle sẽ tạo ra nút thắt cổ chai nghiêm trọng về mặt hiệu năng.

7. Initial Sketch vs AI Suggestion:

Ý tưởng ban đầu	Gợi ý của AI	Quyết định	Lý Do Sửa Đổi / Từ Chối AI
Sử dụng Danh sách liên kết kép thông thường. Dùng if-else để kiểm tra điều kiện biên theo hướng thủ công	Gợi ý nâng cấp lên cấu trúc Circular Linked List	Nhóm giữ nguyên prompt định hướng ban đầu.	Cấu trúc vòng giúp các tính năng Next, Repeat All tự động chạy tuần hoàn, loại bỏ các lệnh if-else rườm rà

Nhóm đề xuất dùng ArrayList để lấy ngẫu nhiên theo chỉ số index.	Hoán vị trực tiếp cấu trúc chuỗi liên kết các nút trên Linked List bằng thuật toán Fisher-Yates , hoặc chuyển đổi liên tục toàn bộ dữ liệu qua lại giữa Linked List và Mảng tạm.	Nhóm từ chối các phương án ban đầu của AI	Vì gây lãng phí tài nguyên và làm tăng độ phức tạp lên $O(n)$. Thay vào đó, bổ sung ArrayList<Song> vào Class PlaylistManager để lưu bản sao tham chiếu.
Nhóm dự định sử dụng con trỏ để dịch ngược tuyến tính (lùi lại một vị trí) ngay trên danh sách phát chính.	AI đề xuất hai phương án: 1. Sử dụng một mảng tĩnh để lưu trữ thứ tự các bài hát đã nhấn. 2. Sử dụng cấu trúc Stack (Ngăn xếp) hoạt động độc lập	Nhóm giữ nguyên prompt định hướng ban đầu và từ chối phương án 1 vì mảng tĩnh giới hạn dung lượng lưu trữ lịch sử.	Nhóm chấp nhận phương án 2 (dùng Stack) vì hành vi dữ liệu tuân thủ nghiêm ngặt nguyên lý LIFO (Last-In-First-Out) nhằm đảm bảo lịch sử chính xác tuyệt đối ngay cả khi danh sách phát đang bị xáo trộn bởi Shuffle.